



4.

Auflage



Mahboubha Gharbi · Arne Koschel
Andreas Rausch · Gernot Starke

Basiswissen für Software- architekten

Ein Überblick

Mahbouba Gharbi · m.gharbi@itech-progress.com

Arne Koschel · akoschel@acm.org

Andreas Rausch · andreas.rausch@tu-clausthal.de

Gernot Starke · gs@gernotstarke.de

Copy Editing: Ursula Zimpfer, Herrenberg

Satz und Herstellung: Frank Heidt

Umschlaggestaltung: Helmut Kraus, www.exclam.de

Druck: www.wp-consult.eu

Artikel-Nr. 077.95735

4., aktualisierte Auflage 2024

Copyright © 2024 dpunkt.verlag GmbH

Wieblinger Weg 17

69123 Heidelberg

Die vorliegende Publikation ist urheberrechtlich geschützt. Alle Rechte vorbehalten. Die Verwendung der Texte und Abbildungen, auch auszugsweise, ist ohne die schriftliche Zustimmung des Verlags urheberrechtswidrig und daher strafbar. Dies gilt insbesondere für die Vervielfältigung, Übersetzung oder die Verwendung in elektronischen Systemen.

Es wird darauf hingewiesen, dass die in der Broschüre verwendeten Soft- und Hardwarebezeichnungen sowie Markennamen und Produktbezeichnungen der jeweiligen Firmen im Allgemeinen warenzeichen-, marken- oder patentrechtlichem Schutz unterliegen.

Alle Angaben und Programme in dieser Broschüre wurden mit größter Sorgfalt kontrolliert. Weder Autorinnen noch Autoren noch Verlag können jedoch für Schäden haftbar gemacht werden, die in Zusammenhang mit der Verwendung dieser Broschüre stehen.

**Mahbouba Gharbi · Arne Koschel · Andreas Rausch ·
Gernot Starke**

Basiswissen für Softwarearchitekten

Ein Überblick

4., aktualisierte Auflage



dpunkt.verlag

Vorwort zur 4. Auflage

Softwarearchitektur bildet – neben motivierten Teams und gutem Management – einen wichtigen Erfolgsfaktor von Softwareprojekten. Sie stellt im Sinne einer systematischen Konstruktion sicher, dass Qualitätsanforderungen wie beispielsweise Erweiterbarkeit, Flexibilität, Performance oder Time-to-Market erfüllt werden können.

Diese Broschüre basiert auf unserem Buch »Basiswissen für Softwarearchitekten« [GK++23] und orientiert sich an der jeweils aktuellen Version des Lehrplans zum »Certified Professional for Software Architecture – Foundation Level« (CPSA-F) des International Software Architecture Qualification Board (ISAQB). Mit der Broschüre erhalten Sie einen Einstieg in die vielschichtige Disziplin der Softwarearchitektur.

Wir als Autorinnen und Autoren arbeiten, lehren und forschen seit vielen Jahren im Bereich des Software & Systems Engineering sowie zur Konstruktion mittlerer und großer IT-Systeme. Wir hoffen, einen Teil unserer Erfahrungen auf diesem Gebiet in unserem Buch und in dieser kompakten Übersicht für Sie als Leserin oder Leser¹ angemessen aufbereitet zu haben.

Wir wünschen Ihnen viel Spaß beim Lesen sowie viel Erfolg bei der Anwendung dieses Wissens in Ihrem Berufsalltag oder, falls Sie sich für eine Schulungsmaßnahme entscheiden, bei einer Prüfung zum CPSA-F.

Mahbouba Gharbi, Arne Koschel, Andreas Rausch, Gernot Starke
Ludwigshafen, Hannover, Clausthal-Zellerfeld, Köln, im Oktober 2023

1. Im weiteren Verlauf der Broschüre wird abwechselnd pro Kapitel die männliche und die weibliche Form verwendet. Wir hoffen, dass sich dadurch alle Leserinnen und Leser angesprochen fühlen.

Einleitung

Softwarearchitektur ist eine sehr junge Disziplin, über deren genauen Umfang und ihre Ausgestaltung in der Informatik trotz vieler Publikationen immer noch viele unterschiedliche Meinungen kursieren. Aufgaben und Verantwortungsbereiche von Softwarearchitekten werden sehr unterschiedlich definiert und in vielen Softwareprojekten ständig neu verhandelt.

Vor diesem Hintergrund haben Anfang 2008 verschiedene Softwarearchitekturexperten aus Wirtschaft und Wissenschaft das »International Software Architecture Qualification Board« als eingetragenen Verein (iSAQB e.V., www.isaqb.org) gegründet. Dessen Ziel ist es, Standards für die Ausbildung und Zertifizierung von Softwarearchitekten zu definieren. Bewusst wird im iSAQB jegliche Hersteller- oder Produktorientierung vermieden. Zertifizierungen auf den unterschiedlichen Stufen Foundation Level, Advanced Level und Expert Level ermöglichen es Softwarearchitekten, sich den Stand ihrer Kenntnisse und Fähigkeiten durch ein anerkanntes Zertifikat bescheinigen zu lassen (siehe Abb. 1).

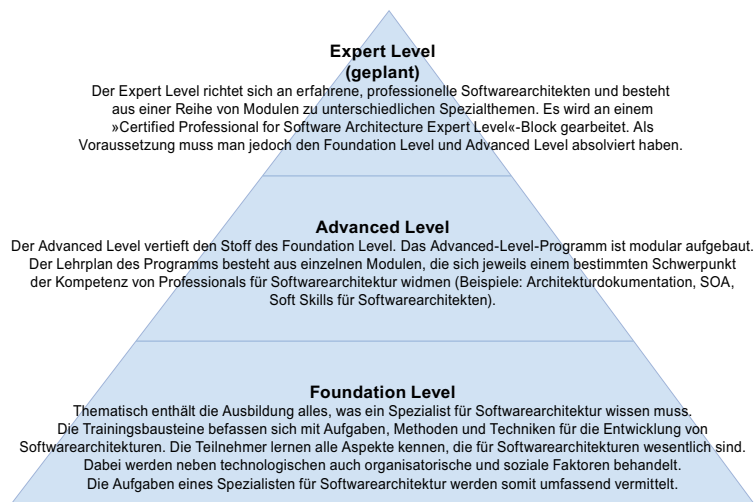


Abb. 1 iSAQB-Zertifizierungsstufen (www.isaqb.org)

Certified Professional for Software Architecture – Foundation und Advanced Level

Der iSAQB hat inzwischen nicht nur die Zertifizierungsrichtlinien für den CPSA Foundation Level, sondern auch für den Advanced Level definiert.

Für den *Foundation Level* wurden die Bereiche, in denen ein Softwarearchitekt über fundiertes Wissen und Fähigkeiten verfügen sollte, im Rahmen eines öffentlich zugänglichen Lehrplans beschrieben [isaqb-lehrplan]. Danach soll angehenden Softwarearchitekten folgendes Spektrum an Inhalten vermittelt werden:

- ❑ der Begriff und die Bedeutung von Softwarearchitektur,
- ❑ die Aufgaben und Verantwortungsbereiche von Softwarearchitekten,
- ❑ die Rolle der Softwarearchitekten in Projekten,
- ❑ State-of-the-Art-Methoden und -Techniken zur Entwicklung von Softwarearchitekturen.

Abbildung 2 zeigt die inhaltliche Struktur und die Gewichtung der einzelnen Bereiche des Lehrplans für den iSAQB Certified Professional for Software Architecture – Foundation Level (CPSA-F).

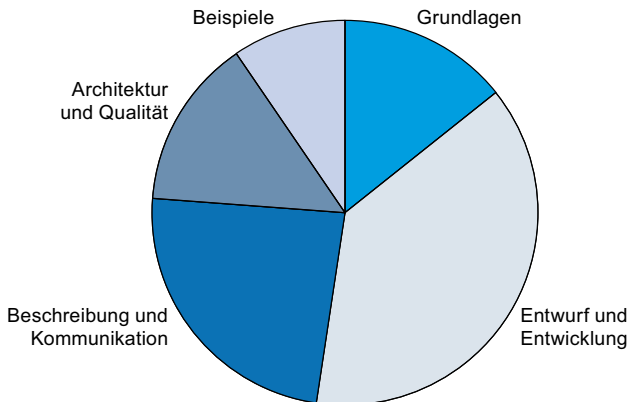


Abb. 2 Struktur des iSAQB-Lehrplans für CPSA-F

Der *Advanced Level* ist modular aufgebaut und besteht aus einzelnen Schulungen, die sich jeweils einem bestimmten Schwerpunkt der Kompetenz eines IT-Professionals widmen:

- ❑ **Methodische Kompetenz:** Wissen und Fähigkeiten im Bereich des systematischen Vorgehens bei IT-Projekten, unabhängig von Technologien.
- ❑ **Technische Kompetenz:** Wissen und Fähigkeiten im Bereich des Einsatzes von Technologien zur Lösung von Entwurfsaufgaben.
- ❑ **Kommunikative Kompetenz:** Wissen und Fähigkeiten im Bereich der Kommunikation, Präsentation, Rhetorik und Moderation zur effektiven Wahrnehmung der Rolle im Softwareentwicklungsprozess.

In dieser kompakten Übersicht erklären wir vor allem wichtige Begriffe und Konzepte der Softwarearchitektur und stellen deren Bezug zu anderen Disziplinen dar. Darauf aufbauend führen wir die grundlegenden Techniken und Methoden für den Entwurf, die Beschreibung und Kommunikation sowie die Qualitätssicherung von Softwarearchitekturen ein. Schließlich betrachten wir die Rolle, Aufgaben, das Umfeld und die Arbeitsumgebung von Softwarearchitekten und deren Einbettung in die umfassende Organisations- und Projektstruktur.

Grundlagen von Softwarearchitekturen

Wir geben in diesem Kapitel eine allgemeine Einführung in die grundlegenden Konzepte von Softwarearchitekturen.

Grundlegende Konzepte von Softwarearchitekturen

Softwarearchitektur

Die **Softwarearchitektur** definiert die grundlegenden Prinzipien und Regeln für die Organisation eines Systems sowie dessen Strukturierung in Bausteinen und Schnittstellen und deren Beziehungen zueinander wie auch zur Umgebung. Dadurch legt sie Richtlinien für den gesamten Systemlebenszyklus, angefangen bei Analyse über Entwurf und Implementierung bis zu Betrieb und Weiterentwicklung, wie auch für die Entwicklungs- und Betriebsorganisation fest.

Schnittstellen

Eine **Schnittstelle** repräsentiert einen wohldefinierten Zugangspunkt zum System oder dessen Bausteinen. Dabei beschreibt eine Schnittstelle die Eigenschaften dieses Zugangspunkts, wie z.B. Attribute, Daten und Funktionen. Ziel ist es, diese Eigenschaften möglichst präzise mit allen notwendigen Aspekten zu definieren, wie z.B. Syntax, Datenstrukturen, funktionales Verhalten, Fehlverhalten, nicht funktionale Eigenschaften, Nutzungsprotokoll der Schnittstelle, Technologien, Randbedingungen und Semantik.

Bausteine

Ein **Baustein** bietet Schnittstellen an, die er im Sinne eines Vertrags garantiert. Diese Garantie gilt unter der Bedingung, dass die von ihm benötigten Schnittstellen im Rahmen einer entsprechenden Konfiguration zur Verfügung gestellt werden. (**Export und Import von Schnittstellen**)

Über die angebotenen und benötigten Schnittstellen kapselt der Baustein die Implementierung dieser Schnittstellen. Daher kann er durch andere Bausteine ersetzt werden, die dieselben Schnittstellen exportieren und gegebenenfalls importieren. (**Kapselung und Austauschbarkeit**)



Und schließlich sind Bausteine die Einheit der hierarchischen (De-)Komposition eines softwareintensiven Systems. Das heißt, ein (Super-)Baustein kann durch eine entsprechende Konfiguration von anderen (Sub-)Bausteinen und deren Beziehungen implementiert werden. Wir sagen dann auch, dieser (Super-)Baustein kapselt die (Sub-)Bausteine. Dabei kann diese Kapsel auch äußere Schnittstellen auf innere delegieren und umgekehrt. So werden die Beziehungen zwischen den Bausteinen definiert. **(Konfiguration und hierarchische (De-)Komposition)**

Ziele und Nutzen

Ziel der Softwarearchitektur ist es, einen Lösungsweg zu finden, der in der Lage ist, die an das System gestellten funktionalen und nicht funktionalen Anforderungen zu erfüllen. Eine gute Softwarearchitektur kann nicht nur Anforderungen erfüllen, sondern macht diese explizit und den Weg von der Anforderung zur Lösung nachvollziehbar. Dies ist aber keine Einbahnstraße, vielmehr spiegelt der Architekturentwurf die Möglichkeit der Realisierbarkeit der Anforderungen und deren Konsequenzen für das Requirements Engineering zurück.

Der Softwarearchitekturentwurf im Überblick

Man kann die einzelnen Aufgaben im Architekturentwurf nicht sinnvoll in eine lineare Reihenfolge bringen. Stattdessen teilen wir den Architekturentwurf in vier gleichberechtigte Tätigkeiten auf:

□ Anforderungen und Randbedingungen analysieren

Zentrale Aufgabe der Architekturanalyse ist es, die Ziele, Randbedingungen sowie die funktionalen und insbesondere die nicht funktionalen Anforderungen aus dem Requirements Engineering im Kontext der anderen umgebenden Bereiche zu untersuchen. Dabei müssen Qualität sowie Flexibilität (Stakeholderin ist offen gegenüber Änderung) und Änderbarkeit der Anforderungen (kann sich durch externe Einflüsse im Laufe der Zeit ändern) überprüft werden.

□ Architektursichten und technische Konzepte entwerfen

Hier wird die Architektur detaillierter ausgearbeitet. Es erfolgt insbesondere auch die sichtenbasierte Beschreibung der unterschiedlichen Architekturebenen wie z.B. der fachlichen und der technischen (vgl. auch **Abschnitt »Beschreibung und Kommunikation von Softwarearchitekturen«** auf Seite 19).

❑ **Architektur- und Entwurfsentscheidungen bewerten**

Die erarbeitete Architektur muss qualitätsgesichert werden. Hier können unterschiedliche Techniken angewendet werden, angefangen von den diversen Reviewtechniken über technische Prototypen und Tests bis hin zu analysierenden und bewertenden Verfahren.

❑ **Umsetzung begleiten und prüfen**

Von ihrer Bedeutung her häufig unterschätzt ist die vierte Aufgabe der Architektin: das Kommunizieren der Softwarearchitektur an alle Projektbeteiligten. Nur wenn alle – von der Entwicklerin bis zur Kundin – die Softwarearchitektur verstanden und akzeptiert haben, wird sie erfolgreich umgesetzt werden können und so ihre erhoffte Wirkung entfalten.

Auf Basis dieser Konzepte sollte auch eine effektive Werkzeugunterstützung aufgebaut werden. Für diese einzelnen Aufgabenbereiche existieren bereits eigenständige Werkzeuglösungen (siehe auch Abschnitt »Werkzeuge für Softwarearchitektinnen« [auf Seite 27](#)).

Aufgabe der Architektin ist es, aus den funktionalen und nicht funktionalen Anforderungen unter Berücksichtigung der Vorgaben und Randbedingungen der umgebenden Bereiche einen Bauplan für das System zu entwickeln. Die Architektin hat dabei eine Vielzahl von Schnittstellen zu fast allen Rollen in einem Softwareentwicklungsprojekt zu bedienen:

❑ **Kommunikations- und Diskussionsplattform**

Anhand der Architektur stellt die Architektin gegenüber dem Requirements Engineer, der Kundin und ggf. der Nutzerin die Machbarkeit der Anforderungen dar. Und sie berät anhand der Softwarearchitektur die Projektleitung bei der Projekt- und Iterationsplanung, unterstützt die Risikoanalyse und -vermeidung und hilft so bei der Definition der Arbeitsstrukturierung und -verteilung.

❑ Design- und Implementierungsplan

Für die Entwicklerin ist die Architektin eine zentrale Ansprechpartnerin. Sie legt die Bausteine des Systems sowie deren Schnittstellen und Interaktionsmuster fest. Die Integration von neuen Technologien und innovativen Lösungsansätzen muss sie vorantreiben und mit den Entwicklerinnen diskutieren. Ebenso unterstützt sie die Testerinnen. Im Idealfall gibt sie sogar Testrahmenbedingungen und konkrete Testfälle vor, insbesondere solche zur Überprüfung der konkreten Architekturziele. Sie hilft bei der Festlegung von Reihenfolgen und Testabhängigkeiten. Schließlich nimmt sie architekturelevante Fehlermeldungen entgegen. Sie ist auch zentrale Ansprechpartnerin für Organisationsrollen, wie z.B. Betrieb, Sicherheitsexpertin und Ähnliches.

Entwurf von Softwarearchitekturen

Es gibt keine vordefinierte Abfolge zwischen den in Abbildung 3 dargestellten vier Tätigkeiten des Entwurfsprozesses:

- ☐ Anforderungen und Randbedingungen analysieren
- ☐ Architektursichten und technische Konzepte entwerfen
- ☐ Architektur- und Entwurfsentscheidungen bewerten
- ☐ Umsetzung begleiten und prüfen

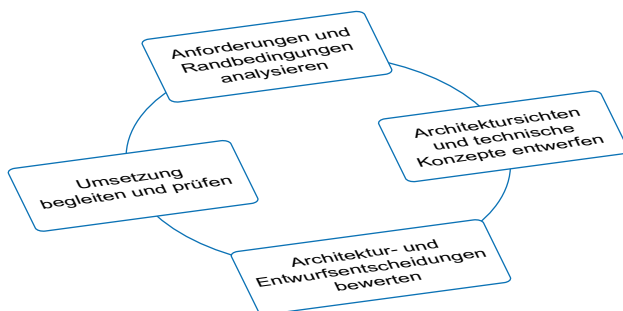


Abb. 3 Die vier Tätigkeiten des Entwurfsprozesses

Im Rahmen des Architekturentwurfs führt der Softwarearchitekt diese vier Tätigkeiten quasi gleichzeitig in beliebiger, für den Projektkontext sinnvoller Abfolge entsprechend den Projektnotwendigkeiten aus. Dabei hält er sich aber an die im Folgenden beschriebenen grundlegenden Prinzipien.

Entwurfsprinzipien und Heuristiken

Als Heuristiken bezeichnet man Verfahren, die helfen sollen, Probleme ressourcensparend zu lösen.

Top-down und bottom-up

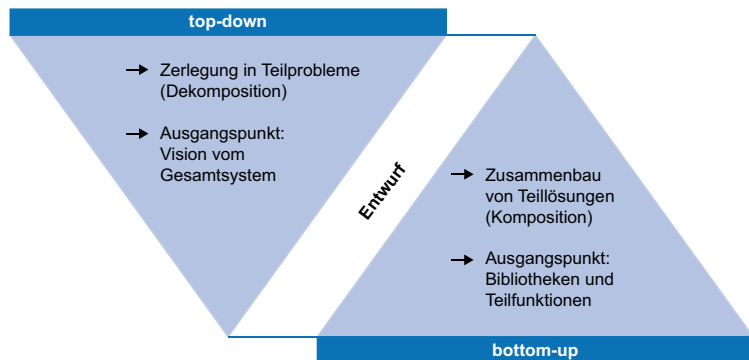


Abb. 4 Top-down und bottom-up

Hierarchische (De-)Komposition

Divide et impera

»Divide et impera« ist lateinisch und bedeutet so viel wie »teile und herrsche« (engl. divide and conquer). Dieses Prinzip findet auch in zahlreichen Algorithmen Anwendung und nutzt die Tatsache, dass der Lösungsaufwand bei vielen Problemen sinkt, wenn man sie in kleinere Teilprobleme zerlegt.

Prinzipien bei der Zerlegung

Die Zerlegung ist ein wichtiger Ansatz, um die Komplexität zu reduzieren [Sta20]. Allerdings gibt es einige zentrale Prinzipien, die dabei beachtet werden sollten, wie z.B. die Kapselung. Ohne die Kapselung können ungewollte Abhängigkeiten zwischen einzelnen Systemteilen entstehen.

So-einfach-wie-möglich-Prinzip

Einfache Strukturen sind verständlicher und dadurch auch leichter veränderbar. Außerdem werden eventuelle Abhängigkeiten leichter gefunden und können einfacher entfernt werden.

Trennung von Verantwortlichkeiten

Das Prinzip der Trennung von Verantwortlichkeiten wird im Englischen »Separation of Concerns« genannt. Allgemein besagt es, dass unterschiedliche Aspekte eines Problems voneinander getrennt und jedes Teilproblem separat für sich behandelt werden soll. Wie viele andere Prinzipien ist es auf das Prinzip »Divide et impera« zurückzuführen.

Konzeptionelle Integrität

Wenn wiederkehrende gleichartige Problemstellungen mit wiederkehrend gleichen Lösungen beantwortet werden, entsteht konzeptionelle Integrität. Dieser Ansatz vereinfacht das Verständnis von Sachverhalten und ist ein Mittel, das »Principle of Least Surprise« zu erreichen. Damit trägt es zur Reduktion von Komplexität bei.

Erwarte Fehler

Niemand ist perfekt und Fehler passieren. Diese Erkenntnis ist in der Softwareentwicklung grundlegend und in vielen zentralen Vorgehensmodellen der Softwareentwicklung wiederzufinden, wie dem Test-Driven Development oder den agilen Vorgehensweisen.

Postel's Law

Der Robustheitsgrundsatz oder auch Postel's Law ist ein konkretes Gesetz für den Umgang mit Fehlern. Es besagt: »Sei streng bei dem, was du tust, und offen bei dem, was du von anderen akzeptierst.«

Schmale Schnittstellen und Information Hiding

Information Hiding

Das Geheimnisprinzip, auch Information Hiding genannt, wurde Anfang der 70er-Jahre von David Parnas beschrieben. Die Komponenten werden als Blackbox betrachtet, der Zugriff auf die innere Struktur wird verwehrt und erfolgt stattdessen über definierte Schnittstellen.

Verwendung von Schnittstellen

Die einzelnen Bausteine und Teilsysteme kommunizieren und kooperieren über die Schnittstellen miteinander. Auch die Kommunikation mit der Außenwelt findet über Schnittstellen statt.

Regelmäßiges Refactoring und Redesign

Software wird in der Regel kontinuierlich weiterentwickelt. Gelingt es nicht, hier regelmäßig aufzuräumen, die aufgrund von Zeitdruck oder Bugfixing entstandenen Ecken und Kanten, hinzugebauten Balkone und Erkerchen sowie die anderen Errungenschaften zurückzubauen und in die grundlegende Architektur zu integrieren, so degeneriert auch eine noch so gute Softwarearchitektur in kürzester Zeit.

Architekturzentrierte Entwicklungsansätze

In diesem Abschnitt werden drei architekturzentrierte Entwicklungsansätze und Konzepte vorgestellt, die Softwarearchitekten heute einsetzen, um Architekturen zu entwerfen und umzusetzen. Die Liste ist keineswegs vollständig.

Domain-Driven Design (DDD)

Beim domänengesteuerten Entwurf (engl. Domain-Driven Design, DDD) handelt es sich um eine Sammlung von Prinzipien und Mustern, die Entwicklern beim Entwurf von Objektsystemen helfen sollen. Der Begriff »Domain-Driven Design« wurde von Eric Evans geprägt [Evans 04]. Auch für ein besseres Verständnis von Microservices ist Domain-Driven Design bedeutend, da es hilft, größere Systeme nach Fachlichkeiten zu strukturieren. Jedes Subsystem soll demnach eine eigene Einheit bilden.

Arten von Domänen

Im DDD kann ein System in folgende Domänen unterteilt werden:

- ☐ Core Domain
- ☐ Generic Subdomain
- ☐ Supporting Subdomain

Die Core Domain enthält die Kernfunktionalität des Systems und beschreibt den Grund für die Existenz des Systems. Sie sollte möglichst nur von den erfahrensten Entwicklern umgesetzt werden.

Die Generic Subdomain enthält Funktionalität, die wichtig für das Geschäft ist, aber nicht Teil der Core Domain ist, wie z.B. Rechnungen erstellen oder Briefe versenden. Sie kann hinzugekauft oder outgesourct werden.

Die Supporting Subdomain enthält unterstützende und untergeordnete Funktionalität und kann auch von weniger erfahrenen Entwicklern übernommen werden. Allerdings sollte sie streng von der Core Domain getrennt werden, z.B. durch Anti-Corruption Layer.

DDD bietet viele weitergehende Möglichkeiten zur Strukturierung der Fachlichkeit wie beispielsweise Subdomänen oder Bounded Context. Es kann verbunden werden mit Ansätzen zur Abbildung der Fachlichkeit in die Technik, wie dem Microservices-Pattern. Zur Integration von verschiedenen Domänen (und den damit verbundenen Teams) gibt es mehrere Möglichkeiten, z.B.:

- ☐ Published Language
- ☐ Open Host Service
- ☐ Anti-Corruption Layer
- ☐ Separate Ways

Globale Analyse

Die globale Analyse ist ein systematischer Ansatz zum Entwurf von Architekturen. Dabei wird auf die Einflussfaktoren auf eine Softwarearchitektur besonderen Wert gelegt. Es wird postuliert, dass Einflussfaktoren Wechselwirkungen und Konflikte untereinander haben und deswegen als Gesamtheit betrachtet werden müssen.

Evolutionäre Architektur

Der Ansatz der evolutionären Architektur sieht vor, eine Architektur zu bauen, die »inkrementelle, geleitete Veränderungen über unterschiedliche Dimensionen hinweg unterstützt« [FPK17].

Inkrementelle Veränderung hat einen hohen Stellenwert in der evolutionären Architektur, da kleine Änderungen die Anpassbarkeit und damit die Evolutionsfähigkeit einer Architektur fördern.

Fitnessfunktionen

In evolutionären Architekturen werden Fitnessfunktionen genutzt, um zu messen, ob eine Eigenschaft der Architektur im gewünschten Umfang vorhanden ist. Damit sind sie nicht nur in der Lage, die Eignung einer Lösung zu beschreiben, sondern auch Lösungen vergleichbar zu machen.

Techniken für einen guten Entwurf

Eine wichtige Herausforderung beim Entwurf von Softwarearchitekturen besteht darin, die gegenseitigen Abhängigkeiten der einzelnen Softwarebausteine vernünftig zu verwalten.

Lose Kopplung

Die Beziehung zwischen den Bausteinen sowie die Stärke dieser Beziehung und der daraus resultierenden Abhängigkeit wird Kopplung genannt.

Nachfolgend sind einige Beispiele für Kopplungsarten aufgeführt:

- ❑ **Aufruf**
Eine Kopplung liegt vor, wenn eine Klasse eine andere Klasse direkt benutzt, indem sie eine Methode der Klasse aufruft.
- ❑ **Erzeugung**
Eine andere Art der Kopplung besteht, wenn ein Baustein einen anderen Baustein erzeugt.
- ❑ **Daten**
Eine weniger starke Kopplung existiert dann, wenn die Klassen über eine globale Datenstruktur oder nur über Methodenparameter kommunizieren.
- ❑ **Ausführungsort**
Eine Kopplung über Hardware besteht, wenn Bausteine in der gleichen Laufzeitumgebung oder der gleichen virtuellen Maschine ablaufen müssen.
- ❑ **Zeit**
Wenn die zeitliche Abfolge von Bausteinaktivitäten eine Rolle spielt, liegt eine Kopplung über die Zeit vor.
- ❑ **Vererbung**
In der Objektorientierung ist eine Unterklasse bereits durch das Erben von Eigenschaften mit ihrer Oberklasse gekoppelt. Der Grad der Kopplung ergibt sich aus der Menge der geerbten Eigenschaften.

Durch die lose Kopplung soll die Komplexität von Strukturen gering gehalten werden.

Hohe Kohäsion

Eine kohärente Klasse löst ein einziges Problem und besitzt eine spezifische Menge an stark zusammenhängenden Funktionen. Je höher die Kohäsion, desto stärker zusammenhängend ist die Zuständigkeit einer Klasse in der Anwendung.

Single-Responsibility-Prinzip

Das Single-Responsibility-Prinzip besagt, dass eine Klasse nicht mehr als einen Grund haben sollte, sie zu ändern. Das Prinzip kann nicht nur für Klassen, sondern für Bausteine auf unterschiedlichsten Abstraktionsebenen angewendet werden.

Offen-geschlossen-Prinzip

Geschlossen heißt, dass das Modul risikolos verwendet werden kann, da sich seine Schnittstelle nicht mehr ändert. Offen hingegen bedeutet, dass das Modul problemlos erweitert werden kann.

Umkehr der Abhängigkeiten

Das Prinzip der Umkehr der Abhängigkeiten (engl. Dependency Inversion) besagt, dass Sie keine direkten Abhängigkeiten, sondern nur Abhängigkeiten von Abstraktionen erlauben sollten. Das führt schließlich dazu, dass die Austauschbarkeit von Bausteinen erleichtert wird.

Abtrennung von Schnittstellen

Bei vielfacher Nutzung einer umfangreichen Schnittstelle kann es sinnvoll sein, diese in mehrere spezifische Schnittstellen zu zerlegen, z.B.:

- ☐ Zerlegung nach semantischem Zusammenhang
- ☐ Zerlegung nach Verantwortungsbereich
- ☐ Zerlegung nach Nutzergruppen

Dahinter steht auch das Single-Responsibility-Prinzip und damit eine einfachere Anpassungsfähigkeit.

Zyklische Abhängigkeiten auflösen

Zyklische Abhängigkeiten erschweren die Wartbarkeit und die Änderbarkeit von Systemen und verhindern eine getrennte Wiederverwendung.

Architekturmuster

Die Klassifikation von Architekturmustern erfolgt nach Frank Buschmann [BM++96] in vier Problemkategorien. Die grundlegende Idee besteht darin, das vom Muster adressierte Problem als Klassifikationsschema zu benutzen:

❑ Adaptierbare Systeme

Muster dieser Kategorie unterstützen die Erweiterung von Anwendungen und ihre Anpassung an sich weiterentwickelnde Technologien und sich ändernde funktionale Anforderungen.

❑ Interaktive Systeme

Muster interaktiver Systeme unterstützen die Strukturierung von interaktiven Softwaresystemen.

❑ Vom Chaos zur Struktur

Muster dieser Kategorie dienen dazu, ein Durcheinander von Komponenten und Objekten zu vermeiden. Insbesondere unterstützen sie bei der sinnvollen Zerlegung einer übergeordneten Aufgabe des Systems in kooperierende Teilaufgaben.

❑ Verteilte Systeme

Muster dieser Kategorie machen Aussagen zu bewährten Formen der Arbeitsteilung und darüber, nach welchem Vorgehen Subsysteme miteinander kommunizieren können.

Herausforderungen verteilter Systeme

Verteilte Systeme bringen neue Herausforderungen in eine Softwarearchitektur, namentlich das Fehlschlagen von Kommunikation, Einschränkungen hinsichtlich der Konsistenz von Persistenz, das »Split-Brain«-Pro-

blem und die Unerreichbarkeit einer eindeutigen zeitlichen Reihenfolge der Ereignisse in einem verteilten System.

Microservices

Ein wichtiges Architekturmuster für verteilte Systeme und deren Integration sind Microservices. Es geht hierbei um die Strukturierung von großen Systemen in kleine fachliche Einheiten. Jeder Microservice soll eine eigene fachliche Einheit repräsentieren. Sie sind weitgehend entkoppelt und laufen selbstständig. Der Ausfall eines Microservice soll die anderen Microservices im Normalfall nicht von der Ausführung ihrer Tätigkeiten abhalten. Nur wenn dies möglich ist, lohnt es sich, das Microservices-Pattern anzuwenden. Im Gegensatz zu Self-contained Systems, die möglichst nicht miteinander reden sollen, können Microservices sowohl synchron als auch asynchron miteinander kommunizieren. Microservices werden separat entwickelt und unabhängig voneinander in Produktion gebracht.

Beschreibung und Kommunikation von Softwarearchitekturen

Dieses Kapitel liefert einen ersten Überblick über das konzeptionelle Rüstzeug zur Beschreibung bzw. Dokumentation von Softwarearchitekturen für die Stakeholderinnen gemäß dem iSAQB-Lehrplan [isaqb-lehrplan].

Sichten und Schablonen

Eine Architektursicht ist eine Repräsentation des Systems aus einer spezifischen Perspektive. Sie hebt wichtige Merkmale eines Betrachtungsgegenstands hervor und abstrahiert von Details, die für diese spezifische Perspektive unbedeutend sind.

Bewährte Sichten nach iSAQB

In der Literatur zur Softwarearchitektur gibt es eine ganze Reihe verschiedener Ansätze zur Beschreibung von Softwarearchitekturen. Viele von ihnen verwenden Sichten. Im Rahmen des iSAQB-Lehrplans werden vor allem vier Sichten behandelt, die als bewährt und besonders praxisrelevant gelten. In Abbildung 5 werden diese vier Sichten und ihr Zusammenspiel dargestellt.

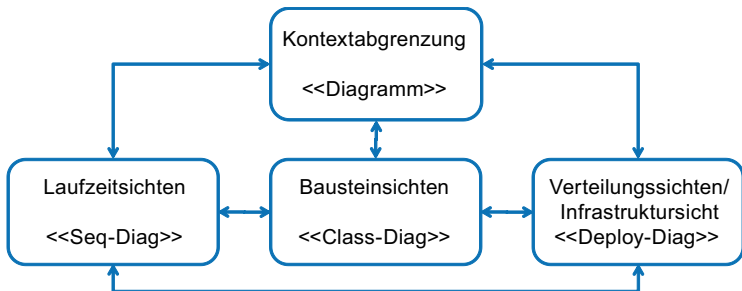


Abb. 5 Bewährte Sichten nach iSAQB

Es handelt sich um:

1. Kontextsicht oder Kontextabgrenzung

Bei der Darstellung der Kontextsicht liegt der Schwerpunkt auf Schnittstellen zu den umliegenden Systemen (Nachbarsysteme). Für die Kontextsicht sind folgende Elemente von Bedeutung:

- ☐ Externe Akteure (Nachbarsysteme und Benutzerinnen)
- ☐ Das zu entwickelnde System selbst
- ☐ *Alle* Schnittstellen zu externen Akteuren (alle Nachbarsysteme bzw. Benutzerinnen)

2. Bausteinsicht

Die Bausteinsicht zeigt die statischen Strukturen eines Softwaresystems. Während der Entwicklung eines Softwaresystems wird seine gewünschte Struktur und nach Fertigstellung seine vorhandene Struktur dargestellt. Ziel dieser Sicht ist es somit, die statische Struktur und die Beziehungen zwischen den Bausteinen der Softwarearchitektur explizit darzustellen.

3. Laufzeitsicht

Die Laufzeitsicht beschreibt das Zusammenwirken der Bestandteile² des Softwaresystems zur Laufzeit. Dabei kommen wichtige Aspekte des Systembetriebs ins Spiel, die beispielsweise den Systemstart, die Laufzeitkonfiguration oder die Administration des Systems betreffen.

4. Verteilungssicht bzw. Infrastruktursicht

Diese Sicht zeigt die Verteilung von Softwareartefakten des Softwaresystems auf Rechnerknoten, Netzwerke usw., also eine Abbildung der Software auf eine reale technische Infrastruktur.

2. Genauer genommen handelt es sich um die *Instanzen* von Bausteinen.

Technische oder querschnittliche Konzepte in Softwarearchitekturen

Technische bzw. querschnittliche Konzepte in Softwarearchitekturen haben – wie der Name andeuten soll – häufig die Eigenschaft, über mehrere Teile oder gar über die gesamte Architektur hinweg zu wirken. Sie können für die gesamte Architekturbeschreibung von Relevanz sein oder nur bestimmte Teile betreffen. Derartige technische oder querschnittliche Konzepte umfassen z. B. Ablaufsteuerung, Fehlerbehandlung, Integration, Internationalisierung, Persistenz und Verteilung.

Übliche Dokumenttypen für Softwarearchitekturen

Zur Beschreibung der in den vorangegangenen Abschnitten dargestellten Architekturinformationen kommt üblicherweise eine Reihe verschiedener Dokumente zum Einsatz. Dieser Abschnitt gibt hierüber einen ersten Überblick.

Zentrale Architekturbeschreibung

Die zentrale Architekturbeschreibung ist (sinnvollerweise) das Kerndokument für eine Softwarearchitektur. Sie enthält, soweit möglich, alle architekturrelevanten Informationen. Hier bietet sich der Einsatz eines passenden Schablonendokuments, also einer Art »Architektur-Dokumentations-Schablone«, an.

Architekturüberblick

Der Architekturüberblick dient als schnell lesbare Kurzfassung (möglichst nicht mehr als 30 Seiten) der zentralen Architekturbeschreibung. Er betrachtet vergleichbare Inhalte, beschränkt sich aber auf die wesentlichen Punkte wie zentrale Sichten, Hauptqualitätsanforderungen und Kernentscheidungen.

Übersichtspräsentation

Eine Übersichtspräsentation ist ein Foliensatz, anhand dessen die Architektur in maximal einer Stunde (technisch) präsentiert werden kann.

Architekturtapete

Mit einer »Architekturtapete« sollen viele Architektur Aspekte in einem Gesamtüberblick dargestellt werden.

Architecture Decision Record

In einem Architecture Decision Record (ADR) wird eine Entscheidung in einem eigenen Dokument oder Abschnitt isoliert betrachtet, mit dem Ziel, eine zugängliche und nachvollziehbare Dokumentation zu erhalten. Der Fokus liegt dabei nicht nur auf dem Ergebnis der Entscheidung, sondern insbesondere auf der Dokumentation der getroffenen Annahmen, Beweggründe und Einflussfaktoren zum Zeitpunkt der Entscheidung.

Dokumentation von externen Schnittstellen

Ein besonderes Augenmerk sollte auf die Dokumentation von insbesondere extern sichtbaren Schnittstellen des Softwaresystems gelegt werden. Diese sind für das Zusammenwirken des Gesamtsystems in seinem Kontext von zentraler Bedeutung.

Softwarearchitekturen und Qualität

Architekturanalyse und deren Methoden sind notwendig für die kontinuierliche Qualitätsprüfung von Architekturen jeder Größe.

Bewertung von Softwarearchitekturen

In Softwareprojekten können Sie prinzipiell zwei Arten von »Dingen« bewerten:

- ☐ Prozesse wie etwa Entwicklungs- oder Betriebsprozesse und
- ☐ Artefakte wie Anforderungen, Quellcode oder andere Dokumente.

Manche dieser Artefakte wie der Quellcode können quantitativ, d.h. in Zahlen, bewertet werden. Andere wiederum entziehen sich der rein zahlenmäßigen Beurteilung und werden qualitativ bewertet, also ihrer Beschaffenheit oder Güte nach.

Qualitative Bewertung

Um die Qualität der Softwarearchitektur bewerten zu können, existieren verschiedene Qualitätsmerkmale und -modelle, die unterschiedliche Aspekte des Softwaresystems im Fokus haben. Diese Qualitätsmodelle bilden die Grundlage für Bewertungen und Analysen, denn sie adressieren die Anforderungen an die Qualitätsmerkmale.

Qualitätsmerkmale

Im Standard ISO/IEC 25010 [ISO/IEC 25010] wurden zu den sechs bisherigen Hauptmerkmalen des ISO/IEC 9126 zwei weitere hinzugefügt: Sicherheit und Kompatibilität. Einige Teilmerkmale sind hinzugekommen und ein paar andere Merkmale wurden umbenannt. Jedes dieser Hauptmerkmale wird noch durch Teilmerkmale verfeinert (siehe Tab. 1).

Funktionale Eignung	☐ Funktionale Vollständigkeit ☐ Funktionale Richtigkeit ☐ Funktionale Angemessenheit
Zuverlässigkeit	☐ Reife ☐ Fehlertoleranz ☐ Wiederherstellbarkeit ☐ Verfügbarkeit
Benutzbarkeit	☐ Erkennbarkeit der Angemessenheit ☐ Bedienbarkeit ☐ Erlernbarkeit ☐ Ästhetik der Benutzeroberfläche ☐ Erreichbarkeit ☐ Schutz des Benutzers vor Fehleingaben
Leistungseffizienz	☐ Zeitverhalten ☐ Verbrauchsverhalten ☐ Kapazität
Sicherheit	☐ Vertraulichkeit ☐ Integrität ☐ Nachweisbarkeit ☐ Verantwortlichkeit ☐ Authentizität
Kompatibilität	☐ Koexistenz ☐ Interoperabilität
Wartbarkeit	☐ Modularität ☐ Wiederverwendbarkeit ☐ Analysierbarkeit ☐ Modifizierbarkeit ☐ Testbarkeit
Übertragbarkeit	☐ Anpassbarkeit ☐ Installierbarkeit ☐ Austauschbarkeit

Tab. 1 Qualitätsmerkmale nach ISO/IEC 25010 [ISO/IEC 25010]

Taktiken und Praktiken

Wie erreicht man nun bestimmte Qualitätsmerkmale? Wichtig ist zuallererst: Es gibt keine allgemeingültige Methode zur Entwicklung von Lösungsstrategien für bestimmte Qualitätsmerkmale. Als Architekt müssen Sie situativ und kontextbezogen passende Maßnahmen parallel zum Entwurf von Sichten und technischen Konzepten entwickeln.

Quantitative Bewertung

Neben der qualitativen gibt es noch die quantitative Bewertung, mit deren Hilfe die Artefakte einer Architektur in Zahlen gemessen werden können. Letztere können gute Hinweise auf strukturelle Veränderungen geben, sofern sie eine Zeit lang konstant gemessen und aufgezeichnet werden. Sie liefern allerdings keine Aussage über die Funktionsfähigkeit und Qualität zur Laufzeit. Ebenso benötigen die Ergebnisse der quantitativen Bewertung einen fachlichen und technischen Kontext, um vergleichbar zu sein.

Quantitative Methoden zur Qualitätssicherung von Architekturen sind z.B.:

- ❑ Metriken zur Vermessung und Bewertung von Architekturaspekten, wie z.B. eingehende und ausgehende Abhängigkeiten oder Komplexität (zyklomatische Komplexität)
- ❑ Analysen, die stärker den Prozess betrachten, wie z.B. Änderungshäufigkeiten
- ❑ Analysen, die die korrekte Umsetzung der Architektur überprüfen, z.B. durch Überprüfung von Architekturregeln im Code
- ❑ Analysen, die das Software-Repository betrachten, wie z.B. Code-dubletten

Architekturanalyse

ATAM-Methode

ATAM steht für *Architecture Tradeoff Analysis Method* und ist ein methodisches Vorgehen zur qualitativen Architekturbewertung. Diese Methode wurde am Software Engineering Institute der Carnegie Mellon

University entwickelt und dient dazu, eine passende Softwarearchitektur für ein Softwaresystem auszuwählen. Die ATAM gilt als führende Methode im Bereich der Architekturbewertung.

Voraussetzungen für ATAM

Benötigt wird:

- Architekt des Systems (oder technischer Ansprechpartner)
- Architekturdokumentation
- Verantwortlicher fachlicher Ansprechpartner oder Auftraggeber

Vorgehen bei der Bewertung

Zu Beginn jeder Architekturbewertung muss der Kunde bzw. Auftraggeber die für diesen Zweck maßgeblichen Stakeholder identifizieren.

Vor der Definition der Bewertungsziele sollte das Bewertungsteam in der Kick-off-Phase den Stakeholdern die Bewertungsmethode kurz vorstellen. Allen Teilnehmern sollte klar sein, dass es um die Ermittlung von Risiken und Nicht-Risiken sowie um mögliche Maßnahmen geht. Der Auftraggeber stellt die geschäftlichen Ziele des Systems vor, das bewertet werden soll.

Anschließend sollte der verantwortliche Architekt die Architektur des Systems kurz erläutern.

Danach sollten die Stakeholder die wesentlichen geforderten Qualitätsmerkmale erarbeiten und hierarchisch in einem Qualitätsbaum anordnen sowie für die wichtigsten Qualitätsziele Szenarien beschreiben, damit das Bewertungsteam mit der eigentlichen Bewertung beginnen kann.

Nach der Analyse der Szenarien sollten alle Entscheidungen in vier Aspekte eingeteilt werden:

Risiken

Die Risiken sind Teile der Architektur, die je nach Verlauf die Erfüllung von Geschäftszielen gefährden und Probleme bereiten können.

❑ Empfindliche Stellen

Bei den sogenannten »empfindlichen Stellen« (engl. Sensitivity Points) der Architektur können geringe Änderungen bereits weitreichende Folgen haben. Dies sind die kritischen Komponenten in einer Architektur für die Erfüllung eines Qualitätsmerkmals.

❑ Kompromisse

Die Kompromisse oder Trade-offs geben an, ob und wie eine Entwurfsentscheidung eventuell mehrere Qualitätsmerkmale wechselseitig beeinflussen kann.

❑ Nicht-Risiken

Welche Szenarien werden auf jeden Fall (d.h. risikolos) erreicht?

Szenarien

ATAM sieht vor, dass die Qualitätsmerkmale durch Szenarien beschrieben werden sollten (szenariobasierte Architekturbewertung). Diese Szenarien beschreiben, wie ein System in bestimmten Situationen reagiert, charakterisieren das Zusammenspiel von Stakeholdern mit dem System und machen die Risiken zur Erreichung dieser Qualitätsmerkmale abschätzbar. Sie werden eingesetzt, um genau zu spezifizieren, was die Projektbeteiligten unter bestimmten Qualitätsmerkmalen wie z.B. »Zuverlässigkeit« verstehen.

Werkzeuge für Softwarearchitektinnen

Verschiedene Kategorien von Werkzeugen unterstützen die Aufgaben von Softwarearchitektinnen. Dieses Kapitel stellt diese Kategorien vor.

Werkzeuge zum Anforderungsmanagement

Anforderungsmanagement wird über den gesamten Lebenszyklus der Systementwicklung durchgeführt, sowohl bei Neuentwicklungen als auch bei Änderungen.

Das Anforderungsmanagement beinhaltet neben der reinen Erhebung von Anforderungen auch Maßnahmen zu deren Verwaltung. Ziel des Anforderungsmanagements ist das gemeinsame Verständnis zwischen Auftragnehmerin und Auftraggeberin über das zu entwickelnde System. Die dabei entstehende Dokumentation dient oft als vertragliche Grundlage für die weitere Umsetzung und sollte daher eine fest vorgegebene Struktur besitzen.

Werkzeuge zur Modellierung

Modellierungswerkzeuge können fachliche und technische Modelle von Software sowie Anforderungs- und Problemdomänen darstellen. Sie helfen dabei, meist grafische Abstraktionen der Realität zu erstellen und zu pflegen.

Damit können solche Werkzeuge beispielsweise beginnend bei (abstrakten) Geschäftsprozessen durch schrittweise Verfeinerung immer detailliertere Darstellungen der jeweiligen Sachverhalte unterstützen.

Werkzeuge zur statischen Codeanalyse

Bei der statischen Analyse wird der Quelltext einer Reihe formaler Prüfungen unterzogen, um die Anwendung nach Auffälligkeiten und Fehlern zu durchsuchen. Dies kann manuell sowie werkzeuggestützt erfolgen.

Werkzeuge können dabei helfen, bestehende Softwaresysteme bezüglich verschiedener Qualitätseigenschaften wie z. B. Komplexität zu bewerten. Werkzeuge zur statischen Codeanalyse können durch das Aufzeigen der Abhängigkeiten auch zur Optimierung der Laufzeiteffizienz genutzt werden. Des Weiteren analysieren sie, ob die Umsetzung den Vorgaben der Architektur entspricht und beispielsweise die Regeln bezüglich zulässiger Abhängigkeiten befolgt werden.

Werkzeuge zur dynamischen Analyse

Werkzeuge zur dynamischen Analyse betrachten das Laufzeitverhalten von Software. Durch Analyse und Vergleich von laufenden Programmen können sie Entwicklerinnen helfen, Problembereiche aufzudecken oder Abläufe zu verstehen. Ziele sind unter anderem:

- ☐ Geschwindigkeitsmessung
- ☐ Zeitmessung bestimmter Systemteile in Relation zu anderen Systemteilen
- ☐ Messung der Speichernutzung
- ☐ Statistische Auswertung (Welche Systemteile werden wie häufig verwendet?)

Werkzeuge zum Konfigurations- und Versionsmanagement

Ein Konfigurationsmanagementwerkzeug soll die Softwarearchitektinnen hauptsächlich bei folgenden Aufgaben unterstützen:

- ☐ Zuordnung und Selektion von Konfigurationselementen zu einer Konfiguration
- ☐ Inventarisierung
- ☐ Rekonstruktion einer Konfiguration

Werkzeuge zum Codemanagement

Codemanagementwerkzeuge sollen Architektinnen und Entwicklerinnen bei der Erstellung, Bearbeitung und dem Verständnis von Quellcode unterstützen. Zu dieser Kategorie von Werkzeugen zählen:

- ❑ Syntaxbezogene Editoren
- ❑ Refactoring-Werkzeuge: Umformung von Quellcode unter Beibehaltung der funktionalen Eigenschaften
- ❑ Debugger
- ❑ Integrierte Entwicklungsumgebungen

Werkzeuge zum Test

Durch die Erstellung automatisierter Unit- und Integrationstests bekommen Softwarearchitektinnen und Entwicklerinnen frühzeitiges Feedback über die Struktur und internen Schnittstellen ihrer Bausteine und deren Zusammenarbeit. Tests sind oftmals die ersten »Benutzer« neu erstellter Bausteine – und können wertvolle Hinweise zu deren Erstellung, Evolution und Verbesserung geben. Testwerkzeuge, u.a. für Unit- und Integrationstests, gehören daher zu dieser Kategorie von Werkzeugen:

- ❑ Unit Tests, z. B. xUnit-Derivate
- ❑ Laufzeittests, etwa Last-/Performance-Tests, Stresstests, Robustheitstests
- ❑ Penetrationstests, Angriffsszenarien
- ❑ Verwaltung von Testfällen, Testdaten, Testergebnissen

Werkzeuge zur Dokumentation

Werkzeuge zur Dokumentation sollen Softwarearchitektinnen und Entwicklerinnen bei der langfristigen Kommunikation von Entscheidungen, Strukturen, Konzepten sowie sonstigen Sachverhalten unterstützen. Zu dieser Kategorie gehören text- und grafikbasierte Werkzeuge zur Erstellung, Pflege oder Generierung von Dokumenten.

Literatur

[BM++96] Buschmann, F.; Meunier, R.; Rohnert, H.; Sommerlad, P.: A System of Patterns: Pattern-Oriented Software Architecture, Volume 1. John Wiley & Sons, 1996.

[Evans 04] Evans, E.: Domain Driven Design. Addison-Wesley, 2004.
<http://www.domaindrivendesign.org>.

[FPK17] Ford, N.; Parsons, R.; Kua, P.: Building Evolutionary Architectures: Support Constant Change. O'Reilly Media, 2017.

[GK++23] Gharbi, M.; Koschel, A.; Rausch, A.; Starke, G.: Basiswissen für Softwarearchitekten. Aus- und Weiterbildung nach iSAQB-Standard zum Certified Professional for Software Architecture – Foundation Level. 5., überarb. u. akt. Aufl., dpunkt.verlag, Heidelberg, 2023.

[isaqb-lehrplan] International Software Architecture Qualification Board e.V. (iSAQB): Lehrpläne.
<https://isaqb-org.github.io/curriculum-foundation/foundation-curriculum-de.pdf>.

[ISO/IEC 25010] ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models.

[Sta20] Starke, G.: Effektive Softwarearchitekturen: Ein praktischer Leitfaden. 9. Aufl., Hanser Verlag, München, 2020.



Mahboub Gharbi ·
Arne Koschel · Andreas
Rausch · Gernot Starke

**Basiswissen für
Softwarearchitekten**
5., überarb. und akt. Auflage
2023, 248 Seiten,
Festeinband
€ 34,90 (D)
ISBN 978-3-86490-984-9



Carola Lilienthal

**Langlebige
Software-Architekturen**
3., überarb. und erweiterte
Auflage
2019, 316 Seiten,
komplett in Farbe, Broschur
€ 34,90 (D)
ISBN 978-3-86490-729-6



Eberhard Wolff

Microservices
2., aktualisierte Auflage
2018, 384 Seiten, Broschur
€ 36,90 (D)
ISBN 978-3-86490-555-1



Eberhard Wolff

**Das Microservices-
Praxisbuch**
2018, 328 Seiten, Broschur
€ 36,90 (D)
ISBN 978-3-86490-526-1



Brendan Burns · Joe Beda ·
Kelsey Hightower ·
Lachlan Evenson

Kubernetes
3., akt. und erweiterte
Auflage
2023, 330 Seiten, Broschur
€ 39,90 (D)
ISBN 978-3-86490-959-7



Vaughn Vernon

**Domain-Driven Design
kompakt**
Aus dem Englischen über-
setzt von Carola Lilienthal
und Henning Schwentner
2017, 158 Seiten,
komplett in Farbe, Broschur
€ 29,90 (D)
ISBN 978-3-86490-439-4



Carola Lilienthal ·
Henning Schwentner

**Domain-Driven
Transformation**
2023, 312 Seiten,
komplett in Farbe, Broschur
€ 34,90 (D)
ISBN 978-3-86490-884-2



Stefan Hofer ·
Henning Schwentner

Domain Storytelling
2023, 254 Seiten,
komplett in Farbe, Broschur
€ 34,90 (D)
ISBN 978-3-86490-958-0



Mahbouba Gharbi · Arne Koschel · Andreas Rausch · Gernot Starke

Basiswissen für Softwarearchitekten

Softwarearchitektur bildet – neben motivierten Teams und gutem Management – einen wesentlichen Erfolgsfaktor für Softwareprojekte. Sie stellt im Sinne einer systematischen Konstruktion sicher, dass Qualitätsanforderungen wie beispielsweise Erweiterbarkeit, Flexibilität, Performance oder Time-to-Market erfüllt werden können.

Diese Broschüre bietet Ihnen einen Einstieg in die vielschichtige Disziplin der Softwarearchitektur. Sie basiert auf dem Buch »Basiswissen für Softwarearchitekten« (ebenfalls erschienen im dpunkt.verlag) und orientiert sich am Lehrplan zum »Certified Professional for Software Architecture – Foundation Level« (CPSA-F) des International Software Architecture Qualification Board (ISAQB).

Die Broschüre richtet sich an Praktiker und Studierende, die einen Einstieg in das Thema Softwarearchitektur suchen.

